Solution Architecture

Date	20 June 2026
Team ID	
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Solution Architecture Diagram:
Based on the template layout, here is how your specific project components map onto the architectural layers

Presentation / Client Layer

- *Web Browser Dashboard Interface*
- *Weather Parameter Input Form & Results Panel*

[Technologies: HTML5, CSS3, JavaScript]

↓↑ (HTTP Requests / Responses)

API Gateway / Load Balancer

- Flask Route Handling & URL Mapping
- Form Input Validation Engine

[Technologies: Python Flask WSGI Router]

↓↑ (Internal App Routing)

Auth Service

- *Session Verification*
- *User Access Controls*

[Technologies: Flask-Login]

Core Logic Service

- *Data Preprocessing (Scaling)*
- *XGBoost Prediction Engine*

[Technologies: Scikit-Learn, XGBoost]

External API Integrations

- *Automated Data Ingestion: Connects to Kaggle/Open Weather APIs to pull real-time historical and live meteorological dataset logs.*
- *Deployment Endpoint: Communicates with IBM Cloud platform runtime services for hosting and environment management.*

[Technologies: Kaggle API, IBM Cloud DevOps API, Requests Library]

↓↑ (Serialized Load / Data Logging)

Data / Storage Layer

• Serialized ML Objects (`model.pkl` & `scaler.pkl`)

• Relational Prediction History Database File

[Technologies: Joblib/Pickle Storage, Python File Logging]

Component Description Table

Here is the completed table formatted to match your project specifications exactly, outlining what each tier does and the precise tech used:

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	*Serves as the user-facing web browser interface. It displays the main project dashboard, presents the meteorological input form fields to the end user, and dynamically displays the localized "Flood Chance" or "No Flood Chance" probability screens.*	HTML5, CSS3, JavaScript
API Gateway	Acts as the operational entry gate for incoming client requests. The layer performs backend URL route mapping, intercepts web form submissions from the client browser, coordinates cross-origin data validation checks, and securely directs clean datasets to lower processing logic engines.	Python / Flask WSGI Routing Engine
Microservices *(Auth & Core)*	*1. Core Logic Engine: Controls operational data adjustments. It applies standard scalar normalization on incoming numbers and passes the scaled arrays directly into the pre-trained, high-accuracy XGBoost classifier to generate predictions.* ***2. Auth Service:*** *Coordinates administrative authorization levels and standard dashboard logging access.*	*XGBoost, Scikit-Learn (Scaler API), Joblib, Flask-Login*
Database	*Serves as the structural persistence layer. It stores the frozen, pre-trained serialization coefficients required by the application backend to continuously evaluate data without retraining, and retains chronological histories of past prediction attempts for reporting.*	*Serialized File Storage Objects (model.pkl / scaler.pkl) via Joblib/Pickle*